

# Laboratório de Concorrência · Aula 10

Lucas Domingues de Souza · RA 24538741-2 · Turma A

Gerado em 09/10/2026, 20:37:05 · tempo registrado no cronômetro: 30:56

## Preparação: Concorrência em cinco minutos

---

**0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.**

Cooperação é quando duas partes dependem uma da outra para avançar, como um motorista esperando o pedestre atravessar ou a cozinha esperando a mesa ser servida. Competição é quando duas partes disputam o mesmo recurso, como duas pessoas tentando usar a mesma caixa de atendimento ao mesmo tempo.

## Prática 1: A condição de corrida

---

**Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?**

Eu espero que o resultado seja menor que 2.000.000, porque as duas threads vão interferir uma na outra. Em geral, o programa imprime um valor abaixo do esperado, pois uma das somas se perde.

**1ª execução (n = 1.000.000)**

1999998

**2ª execução (n = 1.000.000)**

2000000

**3ª execução (n = 1.000.000)**

1999999

**Saída do programa**

```
esperado: 2000000
obtido: 1999998
esperado: 2000000
obtido: 2000000
esperado: 2000000
obtido: 1999999
```

**1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?**

Nem sempre. Os valores não foram iguais nas três execuções. Isso acontece porque o escalonador decide a ordem em que cada thread lê, soma e grava, e como as duas acessam o mesmo dado ao mesmo tempo, uma atualização pode sobrescrever a outra.

**1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.**

Exemplo: A lê contador = 5; B também lê contador = 5 antes de A gravar; A calcula 6 e grava; B calcula 6 e grava por cima do valor de A. O valor final fica 6, e não 7. A soma feita por A se perdeu.

**1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.**

Com  $n = 1000$ , o resultado ficou mais imprevisível e muitas vezes ainda errado, mas em alguns casos pode coincidir com o esperado. Isso é perigoso porque em testes a falha pode não aparecer, mas em produção ela aparece em momentos críticos, gerando inconsistência de dados, saldo errado, pedidos duplicados ou comportamento imprevisível.

## Prática 2: Todas as respostas possíveis (Figura 13.1)

---

### Ordens testadas no simulador

ABABAB → 6

AAABBB → 8

AABBAB → 6

BABBAA → 4

BBBAAA → 7

### Saída do programa

AAABBB → 8

BBBAAA → 12

ABABAB → 6

TOTAL = 4: 6 ordens, por exemplo AABABB

TOTAL = 6: 8 ordens, por exemplo ABBAAB

TOTAL = 8: 6 ordens, por exemplo AAABBB

TOTAL = 12: 1 ordem, por exemplo BBBAAA

### 2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Há 20 ordens possíveis ( $C(6,3) = 20$ ) e 4 resultados diferentes: 4, 6, 8 e 12. O resultado correto 8 aparece em 6 ordens.

### 2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Uma ordem que dá 4 é A A B B A B. Primeiro A lê 3; A calcula 4 e guarda em registrador; B lê 3; B calcula 6 e guarda em registrador; A grava 4 na memória; B grava 6 em cima. A operação de B foi "perdida" no sentido de que o valor que ela calculou sobre o dado compartilhado foi sobrescrito pela outra tarefa, e o resultado final ficou 4 porque o valor de B acabou sendo descartado.

## Prática 3: Corrigindo a corrida

---

### Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

A mais rápida provavelmente é a sem sincronização, mas ela pode dar resultado errado. As versões com `synchronized` e `AtomicInteger` são corretas, mas costumarão ser mais lentas por causa da trava e do custo de exclusão mútua.

### Saída do programa

esperado: 2000000

sem sincronizacao: 1999999 em 6 ms

synchronized: 2000000 em 23 ms

AtomicInteger: 2000000 em 18 ms

### 3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões com `synchronized` e `AtomicInteger` deram o valor correto. Elas são mais lentas porque a thread precisa esperar a exclusão mútua, o que reduz o paralelismo e adiciona custo de travas e sincronização.

### 3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um `AtomicInteger` em cada saldo resolve? Por quê? O que seria preciso?

Não. Um `AtomicInteger` em cada saldo resolve só o incremento do saldo individual, mas não a transação inteira. Para uma transferência bancária, é preciso garantir que a operação de debitar de uma conta e creditar na outra seja atômica como um todo, com lock, transação ou controle de consistência entre as duas contas.

## Prática 4: Impasse (deadlock)

### Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não. As mensagens 'A terminou' e 'B terminou' não vão aparecer, porque há deadlock: A segura X e espera Y, enquanto B segura Y e espera X. Nenhuma consegue avançar, então as duas ficam presas em espera circular. O programa detecta o impasse após 1 segundo e mostra que há threads presas.

### Saídas (antes e depois da correção)

```
IMPASSE: 2 threads presas  
sem impasse
```

### 4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A pegou X e esperava Y. B pegou Y e esperava X. Como cada uma segurava o recurso que a outra precisava, nenhuma podia continuar e ficaram presas em espera circular.

### 4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Se as threads sempre peguem os recursos na mesma ordem, não existe ciclo de espera. Em sistemas reais isso aparece em transferências entre contas, estoque de produtos em diferentes depósitos ou pedidos que acessam vários registros ao mesmo tempo.

## Prática 5: Mensagens em vez de memória compartilhada

### Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores, o tempo deve ficar perto de 600 ms, porque a carga é dividida em 3 filas; com 1 trabalhador, seria praticamente 600 ms também, e com 6 trabalhadores, pode ficar próximo de 300 ms ou até um pouco acima, dependendo do escalonamento.

### Tempo com 1 trabalhador

600 ms

### Tempo com 3 trabalhadores

300 ms

### Tempo com 6 trabalhadores

200 ms

## Saída do programa

```
trabalhador 1 fez a tarefa 1
trabalhador 2 fez a tarefa 2
trabalhador 3 fez a tarefa 3
3 trabalhadores, 6 tarefas de 100 ms: levou 300 ms
```

### 5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, o tempo é cerca de 600 ms porque ele executa as 6 tarefas em sequência. Com 3 trabalhadores, o tempo cai para cerca de 300 ms porque 3 tarefas são feitas em paralelo. Com 6 trabalhadores, o tempo pode cair ainda mais, mas não cai abaixo do tempo do trabalho crítico, porque a carga é limitada pela concorrência e pelo overhead. O programa não precisa de synchronized porque os trabalhadores não compartilham memória: cada goroutine recebe tarefas por um canal e apenas lê e escreve mensagens.

## Saídas (antes e depois da troca)

```
fatal error: all goroutines are asleep - deadlock!
1
```

### 5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque um canal sem buffer bloqueia a operação de envio até que outra goroutine receba. O canal com buffer de 1 resolve porque ele pode guardar uma mensagem e liberar o envio até o próximo recebimento; o modelo passa a funcionar como fila.

## Prática 6: O compilador como guarda

### Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

O programa provavelmente não compila, porque a thread não pode acessar a variável local mutável de main sem ownership explícito. Se usar move, a variável é transferida para a thread e o valor impresso será 0 porque a thread recebe uma cópia do valor original.

## Saídas (o erro e o resultado com move)

```
error[E0373]: closure may outlive the current function, but it borrows `contador`
0
```

## Saída do programa

```
contador = 400000
```

### 6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move, a thread recebe a propriedade da variável, mas o valor do contador original de main foi movido para dentro da closure. A thread incrementa a cópia que ela recebeu, enquanto main continua com o valor original, que é 0. Por isso o resultado impresso é 0.

### 6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro aparece só em tempo de execução, quando o programa roda e os valores ficam inconsistentes. Em Rust, o erro é detectado em compilação, antes da execução. A vantagem é maior segurança e menos bugs de

concorrência. O custo é mais trabalho extra de ownership, move e sincronização explícita.

## Prática 7: Uma thread só: o laço de eventos

---

### Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

Eu acho que a ordem será A, D, C, B, porque o `console.log` normal é síncrono, as promessas são microtarefas e o timer com 0 fica em fila de macrotarefa. Mas o tempo de 300 ms do loop bloqueante faz o B e o E atrasarem.

### Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 304 ms
```

#### 7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem foi A, D, C, B. O `console.log` de A e D são síncronos e executam primeiro. A `Promise.resolve().then()` entra na fila de microtarefas e roda antes de B, que está em uma fila de macrotarefas. O timer de 0 ficou depois porque o código de bloqueio ocupou a thread por 300 ms.

#### 7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer de 10 ms só saiu depois porque a thread estava ocupada com o loop `while` de 300 ms. Isso mostra que um servidor Node.js com cálculo pesado pode atrasar respostas e deixar as requisições pendentes, mesmo que o timer tenha sido agendado para 10 ms.

## Fechamento: Quem impede a corrida?

---

### Java sem sincronização

Ninguém: há corrida

### Java com `synchronized`

O programador, com sincronização

### Go com canais

O projeto: não há memória compartilhada

### Rust com `Arc` e `Mutex`

O programador, com sincronização

### JavaScript

Só existe uma thread

#### 8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode acontecer a condição de corrida: duas pessoas leem que ainda há 1 lugar e ambas confirmam a compra, gerando duas vendas para o mesmo assento. Para evitar, eu usaria sincronização, lock no estoque, transação ou validação no servidor antes de confirmar.

#### 8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi que a concorrência não é só uma questão de velocidade: ela também pode criar resultados inconsistentes, travamentos e bugs que só aparecem em condições específicas.